Spring Data JDBC or Spring Data JPA?

Spring Data offers two main options for integrating applications with a relational database over the JDBC driver: Spring Data JDBC and Spring Data JPA. How to choose between the two? As always, the answer is that it depends on your requirements and specific context.

Spring Data JPA (https://spring.io/projects/spring-data-jpa) is the most-used module in the Spring Data project. It's based on the Java Persistence API (JPA), a standard specification included in Jakarta EE (previously known as Java EE). Hibernate is the most popular implementation. It's a robust and battle-tested object-relational mapping (ORM) framework for managing data persistence in Java applications. Hibernate provides many useful features, but it's also a complex framework. If you're not aware of aspects like persistence context, lazy loading, dirty checking, or sessions, you might face issues that will be hard to debug without a sound familiarity with JPA and Hibernate. Once you know the framework better, you'll appreciate how much Spring Data JPA simplifies things and boosts your productivity. To learn more about JPA and Hibernate, you can check out *High-Performance Java Persistence and SQL* by Vlad Mihalcea (https://vladmihalcea.com) and *Java Persistence with Spring Data and Hibernate* by Ca˘ta˘lin Tudose (Manning, 2022).

Spring Data JDBC (https://spring.io/projects/spring-data-jdbc) is a more recent addition to the Spring Data family. It integrates with relational databases following the domain-driven design (DDD) concepts like aggregates, aggregate roots, and repositories. It's lightweight, simpler, and an excellent choice for microservices where domains are usually defined as bounded contexts (another DDD concept). It gives developers more control over SQL queries and allows the use of immutable entities. Being a simpler alternative to Spring Data JPA, it's not a drop-in replacement for every scenario, since it doesn't provide all the features offered by JPA. I recommend learning both, considering your requirements, and then deciding which module suits the specific scenario better.

I've chosen to cover Spring Data JDBC here for its good fit with cloud native applications and its simplicity. Thanks to the Spring Data common abstractions and patterns, you can easily convert a project from Spring Data JDBC to Spring Data JPA. In the following sections, I'll point out the main differences between the two to give you enough information in case you want to try implementing the same requirements using Spring Data JPA. In the code repository accompanying the book, you'll also find a JPA version of Catalog Service you can use as a reference (Chapter05/05-end/catalog-service-jpa).

5.2.1 Connecting to a database with JDBC